

a) Descripción del Proyecto

Finalidad y objetivos principales

La finalidad de este proyecto es el diseño y desarrollo de una aplicación web de comercio electrónico (e-commerce) especializada en figuras coleccionables **Funko Pop**. El objetivo principal es ofrecer una plataforma funcional donde se integren de forma coherente el desarrollo de software, la gestión de bases de datos y la seguridad en el despliegue.

Objetivos específicos:

- Desarrollar una arquitectura completa **Full-stack**.
- Implementar una **base de datos** relacional robusta con información estructurada.
- Desplegar la solución en la nube utilizando **AWS Academy** y contenedores **Docker**.
- Aplicar estándares de seguridad profesional mediante **HTTPS**.

Descripción funcional general

FunkoStore permite a los usuarios visualizar un catálogo de figuras organizadas por categorías (Marvel, Disney, Anime, etc.). La aplicación se divide en dos áreas principales:

1. **Zona Pública:** Catálogo de productos, buscador y formulario de registro/login.
2. **Zona Privada:** Perfil de usuario, gestión de carrito y consulta de pedidos realizados. La interfaz está desarrollada íntegramente en **inglés**

Requeriments tècnics y funcionals

- **Backend:** JavaDBC
- **Base de Datos:** MySQL o MariaDB integrada mediante Docker.
- **Frontend:** Interfaz dinámica servida por el backend (Thymeleaf o integración de HTML/CSS).
- **Despliegue:** Instancia **EC2 en AWS Academy** gestionada con **Docker Compose**.
- **Seguridad:** login con nginx

Funcionalidades implementadas

- **Catálogo Dinámico:** Los productos se cargan directamente desde la base de datos MySQL.
- **Control de Sesiones:** Implementación de Login/Logout seguro.
- **Carrito de Compra:** Lógica de negocio para añadir productos y calcular el total de la compra.
- **Orquestación con Docker:** Uso de un archivo `docker-compose.yml` que levanta tanto el servidor de aplicaciones como el nodo de base de datos de forma automática.

Descripción del público destinatario

- **Coleccionistas de Funkos:** Usuarios finales que buscan adquirir piezas específicas para sus colecciones.
- **Administradores del sistema:** Perfil encargado de actualizar el catálogo y supervisar los pedidos.

Casos de uso principales

- **Browse Catalog:** El usuario explora los Funkos disponibles filtrando por categorías.
- **User Registration & Login:** El usuario crea una cuenta y accede para poder comprar.
- **Add to Cart:** Selección de productos y gestión de cantidades en la cesta de la compra.
- **Order Summary:** Finalización del proceso de compra y generación de un registro de pedido en la base de datos.

b) Stack Tecnològic

Separació dels components frontend i backend

La separación lógica es clara:

- **Frontend (Capa de Presentación):** Utiliza **HTML5, CSS**. Se encarga de mostrar una buena pagina web a los clientes.
- **Backend (Capa de Negocio y Datos):** Desarrollado en **Java** se conecta a la base de datos con JDBC.
- .

Tecnologies emprades

Tecnologia	Funció
Docker Compose	Orquestración de los contenedores (App + BBDD) para asegurar que el entorno sea idéntico en desarrollo y en AWS.
MySQL / MariaDB	Base de datos relacional para el almacenamiento de productos, usuarios y pedidos.
JAVA	JDBC para hacer funciones y modificar la base de datos.
AWS EC2	Servidor virtual en la nube donde se aloja toda la infraestructura.

Relació dels ports TCP utilitzats i justificació

Para que el sistema funcione y sea accesible, se han configurado los siguientes puertos:

- **Port 443 (TCP):** Utilizado para el tráfico **HTTPS**. Es el puerto estándar para navegación segura y es el único que debería estar abierto al público general en el *Security Group* de AWS.
- **Port 80 (TCP):** Utilizado para **HTTP**. Normalmente se usa para redirigir automáticamente al usuario hacia el puerto 443 (seguro).
- **Port 3306 (TCP):** Puerto interno de **MySQL**. **Justificación:** Este puerto solo está abierto para todo el mundo ya que la aplicación que descargas se tiene que conectar a la misma base de datos.
- **Port 22 (TCP):** Reservado para **SSH**. Permite al desarrollador conectarse a la instancia de AWS para realizar tareas de mantenimiento.
-

Consideracions d'arquitectura i seguretat de xarxa

La arquitectura se ha diseñado siguiendo el principio de **mínimo privilegio**:

- **Aislamiento de Red (Docker Network):** Se ha creado una red virtual privada mediante Docker. El contenedor de MySQL no tiene puertos mapeados hacia el exterior (`ports` en el `compose`), lo que impide ataques externos directos a la base de datos.
- **Seguridad en el Host:** El *Security Group* de la instancia EC2 actúa como un firewall de red, permitiendo solo tráfico en los puertos estrictamente necesarios (80, 443 y 22).
- **Cifrado TLS:** Se implementa un certificado digital para que toda la información sensible (como contraseñas de usuarios) viaje cifrada desde el navegador hasta nuestro servidor en AWS.
- **Persistencia:** Se utilizan **Vómenes de Docker** para asegurar que, aunque los contenedores se detengan o reinicien, los datos de la tienda de Funko Pops no se pierdan.

c) Identitat Visual i Disseny

- Creació d'un logotip associat al projecte:
 - Versió completa (símbol i text)



- Versions amb fons clar i fons fosc



- Versions monocromàtiques (negre i blanc)



- Icona derivada del logotip



- Versió tipogràfica

Funko Store

- Guia d'estil del lloc web
 - Paleta de colors



Tipografies

Amiri, Arial y Calibri

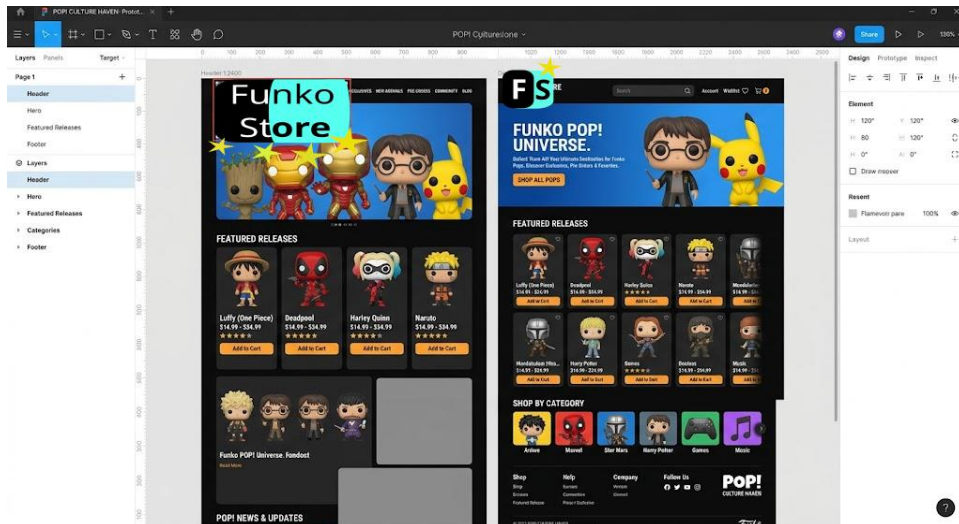
justificació de les decisions del disseny

Negro: transmite solidez y profesionalidad.

Turquesa claro: aporta frescura y modernidad, destacando parte del texto para llamar la atención del cliente.

Estrellas amarillas: simbolizan diversión y coleccionismo, elementos clave de los productos Funko

Prototipat de la interfície mitjançant Figma



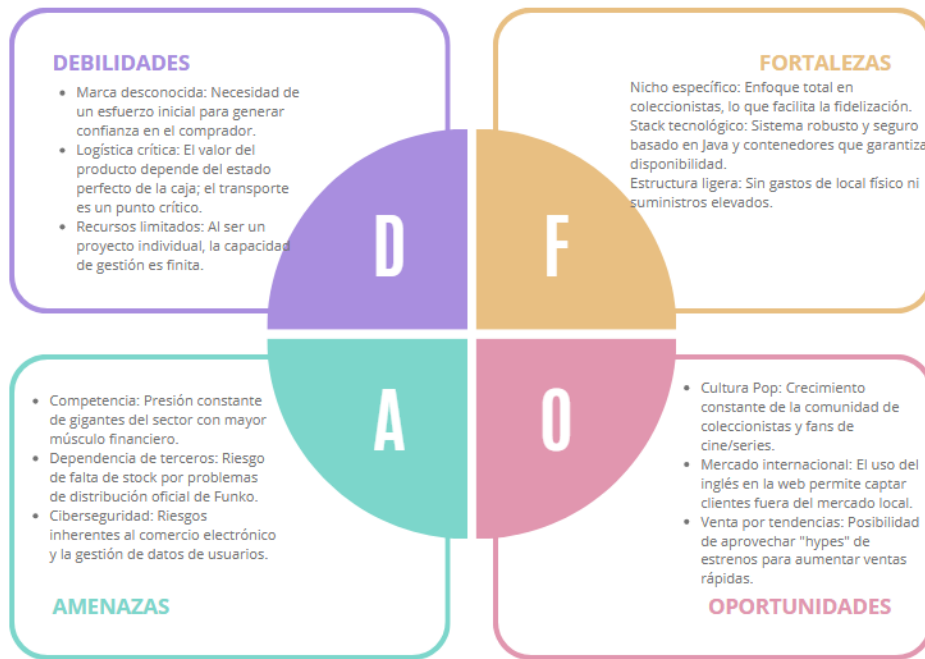
d) Pla de Negoci

Breve estudio de la viabilidad económica

Para que el proyecto sea viable, debemos analizar los costes de infraestructura frente a los ingresos potenciales.

- **Costes de Infraestructura (IT):** Al utilizar **AWS Academy**, el coste de computación es cero durante la formación. En un entorno real, una instancia EC2 pequeña y una base de datos RDS tendrían un coste aproximado de **15-30€/mes**.
- **Costes de Producto:** El margen de beneficio en los Funko Pops suele rondar el **30-40%**. Comprando unidades a distribuidores por 9€ para venderlas a 14,95€.
- **Punto de equilibrio:** Vendiendo aproximadamente **10-15 unidades al mes**, se cubrirían los gastos operativos de la plataforma web.
- **Escalabilidad:** Al estar basado en **Docker**, si la tienda crece, podemos escalar los recursos en la nube de forma rápida sin necesidad de programar todo de nuevo.

DAFO



CAME



e) Infraestructura i Desplegament

Registre o adquisició d'un domini DNS (Simulació)

Para profesionalizar el proyecto, se ha realizado la simulación de adquisición del dominio **funkopopsenia.es**.

funkopopsenia.es	Disponible	25€ 1€	REGISTRAR
------------------	------------	--------	-----------

- **Justificación:** Se ha elegido la extensión .es por ser la más adecuada para el mercado objetivo inicial (España) y por su coste competitivo (1€ en oferta).

Desplegament en AWS Academy utilitzant EC2 i Docker

El despliegue se basa en una arquitectura de contenedores para garantizar la portabilidad:

- **Instancia EC2:** Se ha levantado una instancia de tipo t2.micro (capa gratuita) con una distribución de Linux (Ubuntu/Amazon Linux).
- **Docker Compose:** Se utiliza para orquestar dos servicios principales:
- **Contenedor BBDD:** Ejecuta la imagen oficial de MySQL.
- **Beneficio:** Esta configuración permite que, con un solo comando (`docker-compose up -d`), toda la tienda esté operativa en pocos segundos en el servidor de Amazon.
-

Instal·lació i configuració del certificat digital

Para cumplir con el requisito de seguridad y el protocolo **HTTPS**:

- **Certificado:** Se ha utilizado un certificado emitido por mi.

Gestió del codi font mitjançant un repositori Git públic

Todo el ciclo de vida del desarrollo se ha gestionado con **Git**, alojando el código en un repositorio público (GitHub/GitLab). Esto facilita el control de versiones, el trabajo colaborativo y la integración continua.

Descripció de l'estructura del repositori Git

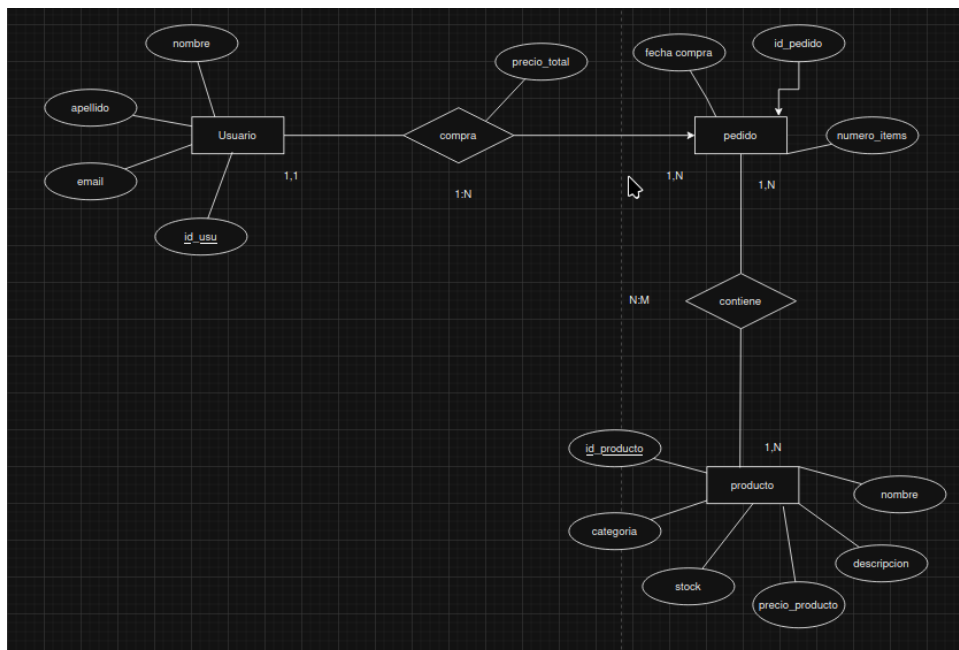
El repositorio se ha organizado siguiendo las mejores prácticas para proyectos:

/src: Contiene todo el código fuente de la aplicación (Java).

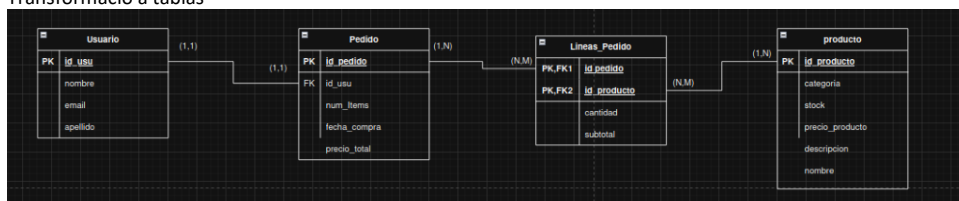
- **src/main/java:** Lógica del backend.
- **src/main/resources:** Plantillas HTML, archivos CSS.
- **/sql:** Scripts de creación de la base de datos y carga de datos iniciales (los Funkos de prueba).
- **Dockerfile:** Instrucciones para empaquetar la aplicación Java en una imagen de contenedor.
- **docker-compose.yml:** Archivo de configuración para levantar la aplicación y la base de datos de forma conjunta.
- **README.md:** Guía rápida en inglés con las instrucciones de instalación y despliegue del proyecto.

BASE DE DATOS

MODELO E-R



Transformació a tablas



SQL

CREATE DATABASE tienda;

USE tienda;

-- USUARIO

```
CREATE TABLE Usuario (  
    id_usu INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    apellido VARCHAR(100),  
    email VARCHAR(150) UNIQUE NOT NULL  
);
```

-- PRODUCTO

```
CREATE TABLE Producto (  
    id_producto INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(150) NOT NULL,  
    descripcion TEXT,  
    categoria VARCHAR(100),  
    stock INT DEFAULT 0,  
    precio_producto DECIMAL(10,2) NOT NULL  
);
```

-- PEDIDO

```
CREATE TABLE Pedido (  
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
    id_usu INT NOT NULL,  
    num_items INT DEFAULT 0,  
    fecha_compra DATETIME DEFAULT CURRENT_TIMESTAMP,  
    precio_total DECIMAL(10,2) DEFAULT 0,  
  
    FOREIGN KEY (id_usu) REFERENCES Usuario(id_usu)  
        ON DELETE CASCADE  
);
```

-- LINEAS_PEDIDO (tabla intermedia N:M)

```
CREATE TABLE Lineas_Pedido (  
    id_pedido INT,  
    id_producto INT,  
    cantidad INT NOT NULL,
```

subtotal DECIMAL(10,2) NOT NULL,

PRIMARY KEY (id_pedido, id_producto),

FOREIGN KEY (id_pedido) REFERENCES Pedido(id_pedido)
ON DELETE CASCADE,

FOREIGN KEY (id_producto) REFERENCES Producto(id_producto)
ON DELETE CASCADE

);

f) Política de Còpies de Seguretat

- Respaldo automático con **Cron**.
- Scripts en **Bash**.
- Almacenamiento seguro.
- Registro de ejecuciones.

```
#  
# m h dom mon dow   command  
0 2 */4 * * tar -czf /backups/proyecto_$(date +%F).tar.gz /home/ubuntu
```